

Factory Method Design Pattern — Bilingual Study Document (English + Hindi)

Inspired by [algomaster.io/learn/lld/factory-method](https://www.algomaster.io/learn/lld/factory-method) — concepts and structure preserved, written in original wording with self-authored PHP examples

Study Companion · Updated July 2026

Factory Method Design Pattern

Note on sourcing / स्रोत पर टिप्पणी: AlgoMaster.io is a paid interview-prep course, and its code samples sit behind a subscription (its Factory Method lesson's implementation sections show section headers with the actual code stripped out in a plain fetch). This document follows the same topic order and teaching structure as their Factory Method lesson — the notification-sending problem → a Simple Factory first attempt → the full Factory Method → a food-delivery-platform analogy → the workflow → step-by-step implementation → a document-export practical example — but every explanation below is written fresh in my own words, and all PHP code is original.

AlgoMaster.io एक पेड (paid) इंटरव्यू-प्रेप कोर्स है, और इसके कोड सैंपल सब्सक्रिप्शन (subscription) के पीछे हैं (इसके Factory Method पाठ के इम्प्लीमेंटेशन सेक्शन्स में सिर्फ हेडिंग्स दिखीं, असली कोड गायब था)। यह दस्तावेज़ उनके Factory Method पाठ जैसा ही विषय-क्रम और टीचिंग स्ट्रक्चर फ़ॉलो करता है — नोटिफ़िकेशन भेजने की समस्या → एक Simple Factory पहला प्रयास → पूरा Factory Method → एक फूड-डिलीवरी-प्लेटफ़ॉर्म उपमा → वर्कफ़्लो → चरण-दर-चरण इम्प्लीमेंटेशन → एक डॉक्यूमेंट-एक्सपोर्ट व्यावहारिक उदाहरण — लेकिन नीचे दी गई हर व्याख्या मेरे अपने शब्दों में ताज़ा लिखी गई है, और सारा PHP कोड ओरिजिनल है।

Overview

The **Factory Method Design Pattern** is a creational pattern that provides a way to create objects inside a base class, while letting subclasses decide exactly which concrete type actually gets created.

Factory Method डिज़ाइन पैटर्न एक क्रिएशनल (creational) पैटर्न है, जो एक बेस क्लास के अंदर ऑब्जेक्ट्स बनाने का एक तरीका देता है, जबकि सबक्लासेज़ को यह तय करने देता है कि असल में कौन-सा कॉन्क्रीट (concrete) टाइप बनाया जाए।

It earns its place in three situations especially: when the exact type needed isn't known until the program is actually running; when creation logic is complex, repetitive, or deserves to be encapsulated away from the code that uses the object; and when you specifically want new types to be added without editing code that already works — the Open/Closed Principle, stated as a design goal rather than just a principle to recite.

यह खासतौर पर तीन स्थितियों में अपनी जगह बनाता है: जब सही टाइप प्रोग्राम असल में चलने तक पता ही नहीं होता; जब क्रिएशन लॉजिक जटिल, दोहराया जाने वाला, या ऐसा हो जिसे उस कोड से अलग रखा जाना चाहिए जो ऑब्जेक्ट का इस्तेमाल करता है; और जब आप खासतौर पर चाहते हैं कि नई टाइप्स, पहले से काम कर रहे कोड में बदलाव किए बिना जोड़ी जा सकें — ओपन/क्लोज़्ड प्रिंसिपल (Open/Closed Principle), एक याद रखने वाले नियम के बजाय एक डिज़ाइन लक्ष्य के तौर पर।

The natural first instinct, when several similar-but-different objects are needed, is a conditional — an `if / else` or `switch` deciding which class to build. That instinct is reasonable at first. The trouble starts once the application grows: this approach stiffens, gets harder to test, and ties the code tightly to specific classes it should ideally not need to know about.

जब कई एक जैसे-मगर-अलग ऑब्जेक्ट्स की ज़रूरत हो, तो सबसे पहला और स्वाभाविक खयाल एक शर्त (conditional) का आता है — एक `if / else` या `switch`, जो तय करे कि कौन-सी क्लास बनानी है। शुरुआत में यह खयाल वाजिब लगता है। मुश्किल तब शुरू होती है जब ऐप्लिकेशन बढ़ता है: यह तरीका अकड़ (stiffen) जाता है, टेस्ट करना मुश्किल हो जाता है, और कोड को उन खास क्लासेज़ से कसकर जोड़ देता है जिन्हें आदर्श रूप से जानने की उसे ज़रूरत ही नहीं होनी चाहिए।

Factory Method exists precisely to let you create these different objects without tying your code tightly to their specific classes.

Factory Method ठीक इसीलिए मौजूद है, ताकि आप इन अलग-अलग ऑब्जेक्ट्स को बना सकें, बिना अपने कोड को उनकी खास क्लासेज़ से कसकर जोड़े।

1. The Problem: Sending Notifications

Picture building a web application that sends notifications. At the very start, there's only one kind: email. One class handles it, and the rest of the service simply builds that object and calls its send method.

एक ऐसा वेब ऐप्लिकेशन बनाने की कल्पना कीजिए जो नोटिफिकेशन्स भेजता है। बिल्कुल शुरुआत में, सिर्फ़ एक ही तरह का नोटिफिकेशन है: ईमेल। एक क्लास इसे संभालती है, और बाक़ी सर्विस बस उस ऑब्जेक्ट को बनाती है और उसका send मेथड कॉल करती है।

Then a new requirement arrives: SMS notifications too. So a new class gets added, and the service that creates notifications grows an `if` branch to build an SMS object when needed, and send that as well. Manageable, if a little more complex.

फिर एक नई ज़रूरत आती है: SMS नोटिफिकेशन्स भी चाहिए। तो एक नई क्लास जोड़ी जाती है, और नोटिफिकेशन्स बनाने वाली सर्विस में एक `if` शाखा (branch) बढ़ जाती है, जो ज़रूरत पड़ने पर एक SMS ऑब्जेक्ट बनाए और उसे भी भेजे। थोड़ा और जटिल, पर संभालने लायक।

A few weeks later, push notifications for mobile are needed. Then Slack alerts. Then WhatsApp. Every one of these adds one more branch to the same growing block of conditionals.

कुछ हफ़्तों बाद, मोबाइल के लिए पुश (push) नोटिफिकेशन्स चाहिए। फिर Slack अलर्ट्स। फिर WhatsApp। इनमें से हर एक, शर्तों (conditionals) के उसी बढ़ते हुए ब्लॉक में एक और शाखा जोड़ देता है।

At this point the notification-creating code has turned into something like a control tower — it's responsible for building every kind of notification, knowing how each one internally works, and deciding which one to build based on a type flag. Here's what's wrong with that shape specifically: every new channel means editing that same core logic again; testing gets awkward because creation logic and usage logic are tangled together; and it plainly violates the Open/Closed Principle — the class isn't open for extension without being modified every single time.

इस बिंदु पर, नोटिफिकेशन बनाने वाला कोड एक तरह के कंट्रोल टावर जैसा बन चुका है — यह हर तरह का नोटिफिकेशन बनाने, हर एक के अंदर कैसे काम करता है यह जानने, और एक टाइप फ़्लैग के आधार पर कौन-सा बनाना है यह तय करने की ज़िम्मेदारी उठाता है। इस खास बनावट में यही समस्या है: हर नया चैनल (channel) मतलब वही मूल लॉजिक फिर से एडिट करना; टेस्टिंग अजीब हो जाती है क्योंकि क्रिएशन लॉजिक और उपयोग लॉजिक आपस में उलझे होते हैं; और यह साफ़ तौर पर ओपन/क्लोज़्ड प्रिंसिपल का उल्लंघन करता है — क्लास हर बार बदले बिना एक्सटेंशन के लिए खुली नहीं है।

2. Simple Factory: A First Attempt

Before reaching for the full Factory Method pattern, there's a common, useful intermediate step: pull the creation logic out into its own class. This is called a **Simple Factory** — not a formal Gang-of-Four pattern, but one of the most practical refactoring moves that actually shows up in real codebases.

पूरे Factory Method पैटर्न तक पहुँचने से पहले, एक आम और उपयोगी बीच का क़दम है: क्रिएशन लॉजिक को एक अलग क्लास में निकाल लेना। इसे **Simple Factory** कहा जाता है — यह कोई औपचारिक (formal) Gang-of-Four पैटर्न नहीं है, लेकिन असली कोडबेसेज़ में सबसे व्यावहारिक रीफ़ैक्टरिंग क़दमों में से एक है।

The idea is simple: one class whose only job is centralizing and encapsulating object creation. The notification service no longer needs to know which concrete class to build — it just asks the factory. Every bit of creation logic now lives in one place, and the service that uses notifications gets noticeably cleaner: it only *uses* the notification, it no longer *constructs* it.

विचार सीधा है: एक क्लास, जिसका इकलौता काम है ऑब्जेक्ट क्रिएशन को केंद्रीकृत (centralize) और एनकैप्सुलेट करना। नोटिफ़िकेशन सर्विस को अब यह जानने की ज़रूरत नहीं कि कौन-सी कॉन्क्रीट क्लास बनानी है — वह बस फ़ैक्टरी से माँगती है। क्रिएशन लॉजिक की हर बिट अब एक ही जगह रहती है, और नोटिफ़िकेशन इस्तेमाल करने वाली सर्विस काफ़ी साफ़-सुथरी हो जाती है: वह सिर्फ़ नोटिफ़िकेशन का इस्तेमाल करती है, अब उसे बनाती नहीं।

But as the product keeps growing and new notification types keep arriving, something starts to feel off again: the Simple Factory begins looking eerily like the bloated code it was meant to replace. Every new type still means editing that same factory's `switch` or `if/else` chain — which isn't especially Open/Closed either. The system is better, genuinely, but it's still not open to extension without modification; creation decisions are still hard-coded and centralized in one place. What's still missing is giving each notification type its own responsibility for knowing how to create itself — and that's precisely the gap Factory Method closes.

लेकिन जैसे-जैसे प्रोडक्ट बढ़ता रहता है और नए नोटिफ़िकेशन टाइप्स आते रहते हैं, कुछ फिर से गड़बड़-सा महसूस होने लगता है: Simple Factory उसी बोज़िल (bloated) कोड जैसी दिखने लगती है जिससे बचने के लिए इसे बनाया गया था। हर नया टाइप अब भी उसी फ़ैक्टरी की `switch` या `if/else` चेन को एडिट करने का मतलब रखता है — जो ख़ास तौर पर ओपन/क्लोज़्ड भी नहीं है। सिस्टम वाक़ई बेहतर है, लेकिन अब भी बिना बदलाव के एक्सटेंशन के लिए खुला नहीं है; क्रिएशन के फ़ैसले अब भी हार्ड-कोडेड और एक ही जगह केंद्रीकृत हैं। जो अब भी नहीं है, वह है हर नोटिफ़िकेशन टाइप को अपनी खुद की ज़िम्मेदारी देना कि वह खुद को कैसे बनाए — और यही वह कमी है जिसे Factory Method भरता है।

3. What Is Factory Method?

The Factory Method pattern hands the idea of object creation off to subclasses. Instead of one central factory deciding everything, the responsibility is delegated to specialized classes that each know exactly what they need to produce.

Factory Method पैटर्न, ऑब्जेक्ट क्रिएशन के विचार को सबक्लासेज़ को सौंप (hand off) देता है। एक केंद्रीय फैक्टरी द्वारा सब कुछ तय करने के बजाय, ज़िम्मेदारी उन खास (specialized) क्लासेज़ को सौंपी जाती है, जिनमें से हर एक को ठीक-ठीक पता होता है कि उसे क्या बनाना है।

In simpler terms: each subclass defines its own way of instantiating an object; the base class defines a common interface for creating that object, without knowing what the object actually is; and the base class typically also defines shared behavior that uses whatever object gets created.

सीधे शब्दों में: हर सबक्लास अपना खुद का तरीका तय करती है कि ऑब्जेक्ट कैसे बनाना है; बेस क्लास उस ऑब्जेक्ट को बनाने के लिए एक कॉमन इंटरफ़ेस डिफ़ाइन करती है, बिना यह जाने कि वह ऑब्जेक्ट असल में है क्या; और बेस क्लास आमतौर पर एक साझा व्यवहार (shared behavior) भी डिफ़ाइन करती है, जो जो भी ऑब्जेक्ट बने, उसका इस्तेमाल करता है।

The creation logic, in other words, becomes decentralized — spread across specialized creators instead of sitting in one central switch statement.

दूसरे शब्दों में, क्रिएशन लॉजिक विकेंद्रीकृत (decentralized) हो जाता है — एक केंद्रीय switch स्टेटमेंट में बैठने के बजाय, विशेष क्रिएटर्स में फैल जाता है।

A real-world analogy: think of a food-delivery platform. If it were designed like a Simple Factory, there would be one centralized kitchen deciding whether to cook pizza, sushi, or burgers for every order. With Factory Method, each restaurant — the pizza place, the sushi bar, the burger joint — has its own kitchen and knows exactly how to prepare its own food; the platform simply asks the right kitchen to handle each order.

एक असल-दुनिया की उपमा: एक फ़ूड-डिलीवरी प्लेटफ़ॉर्म की कल्पना कीजिए। अगर यह एक Simple Factory की तरह डिज़ाइन किया गया हो, तो एक केंद्रीय रसोई (kitchen) यह तय करेगी कि हर ऑर्डर के लिए पिज़्ज़ा बनाना है, सुशी या बर्गर। Factory Method के साथ, हर रेस्तराँ — पिज़्ज़ा प्लेस, सुशी बार, बर्गर जॉइंट — अपनी खुद की रसोई रखता है और ठीक-ठीक जानता है कि अपना खाना कैसे तैयार करना है; प्लेटफ़ॉर्म बस हर ऑर्डर को सही रसोई तक भेज देता है।

The four participants:

Product (e.g., `Notification`) — the interface every concrete product must implement. The rest of the system works only against this interface, never against a specific concrete class.

Product (जैसे `Notification`) — वह इंटरफ़ेस जिसे हर कॉन्क्रीट प्रोडक्ट को इम्प्लीमेंट करना होता है। बाक़ी सिस्टम सिर्फ़ इसी इंटरफ़ेस के खिलाफ़ काम करता है, कभी किसी खास कॉन्क्रीट क्लास के खिलाफ़ नहीं।

ConcreteProduct (e.g., `EmailNotification`) — the actual classes implementing `Product`, each with its own internal behavior (an email notification talks to an SMTP server, an SMS notification talks to a telephony API) while sharing the same method signature.

ConcreteProduct (जैसे `EmailNotification`) — वे असली क्लासेज़ जो `Product` को इम्प्लीमेंट करती हैं, हर एक का अपना आंतरिक व्यवहार होता है (एक ईमेल नोटिफिकेशन SMTP सर्वर से बात करता है, एक SMS नोटिफिकेशन एक टेलीफ़ोनी API से) जबकि सभी एक ही मेथड सिग्नेचर साझा करते हैं।

Creator (e.g., `NotificationCreator`) — an abstract class declaring the factory method, and usually also containing shared logic that uses whatever product the factory method returns. The Creator defines the *workflow*; the subclasses fill in the *detail*.

Creator (जैसे `NotificationCreator`) — एक ऐब्सट्रैक्ट क्लास जो factory मेथड डिक्लेयर करती है, और आमतौर पर साझा लॉजिक भी रखती है, जो factory मेथड द्वारा लौटाए गए किसी भी प्रोडक्ट का इस्तेमाल करता है। Creator वर्कफ़्लो डिफ़ाइन करता है; सबक्लासेज़ बारीकियाँ भरती हैं।

ConcreteCreator (e.g., `EmailNotificationCreator`) — subclasses of Creator overriding the factory method to return one specific ConcreteProduct, one creator paired with exactly one product type.

ConcreteCreator (जैसे `EmailNotificationCreator`) — Creator की सबक्लासेज़, जो factory मेथड को ओवरराइड करके एक खास ConcreteProduct लौटाती हैं, हर creator बिल्कुल एक प्रोडक्ट टाइप के साथ जोड़ा हुआ।

4. How It Works

Step 1 — The client selects a Creator. Based on configuration, user input, or business logic, the client instantiates the appropriate `ConcreteCreator` — say, `EmailNotificationCreator`, if an email notification is what's needed.

चरण 1 — क्लाइंट एक Creator चुनता है। कॉन्फिगरेशन, यूज़र इनपुट, या बिज़नेस लॉजिक के आधार पर, क्लाइंट उपयुक्त `ConcreteCreator` को इंस्टैंशिएट करता है — जैसे, `EmailNotificationCreator`, अगर एक ईमेल नोटिफिकेशन चाहिए।

Step 2 — The client calls a method on the Creator. Typically the high-level operation — `send()` — which lives on the abstract Creator class itself.

चरण 2 — क्लाइंट Creator पर एक मेथड कॉल करता है। आमतौर पर हाई-लेवल ऑपरेशन — `send()` — जो खुद ऐब्स्ट्रैक्ट Creator क्लास पर रहता है।

Step 3 — The Creator calls the factory method. Inside `send()`, the Creator calls `createNotification()`; since the Creator is abstract, this dispatches to whichever `ConcreteCreator`'s override is actually running.

चरण 3 — Creator factory मेथड को कॉल करता है। `send()` के अंदर, Creator `createNotification()` कॉल करता है; चूंकि Creator ऐब्स्ट्रैक्ट है, यह उसी `ConcreteCreator` के ओवरराइड तक पहुँचता है जो असल में चल रहा है।

Step 4 — The ConcreteCreator returns a ConcreteProduct.

`EmailNotificationCreator::createNotification()` returns a new `EmailNotification`; the Creator receives it typed only as the `Notification` interface.

चरण 4 — ConcreteCreator एक ConcreteProduct लौटाता है।

`EmailNotificationCreator::createNotification()` एक नया `EmailNotification` लौटाता है; Creator को यह सिर्फ `Notification` इंटरफ़ेस के रूप में मिलता है।

Step 5 — The Creator uses the product. The Creator calls `send()` on the product it just received; the correct concrete behavior runs, without the Creator ever having needed to know which concrete class it was holding.

चरण 5 — Creator प्रोडक्ट का इस्तेमाल करता है। Creator अभी-अभी मिले प्रोडक्ट पर `send()` कॉल करता है; सही कॉन्क्रीट व्यवहार चलता है, बिना Creator को कभी यह जानने की ज़रूरत पड़े कि वह किस कॉन्क्रीट क्लास को पकड़े हुए था।

5. Implementing Factory Method (in PHP)

The implementation below is original code written for this study document, following the same six steps: the Product interface, the ConcreteProducts, an abstract Creator, the ConcreteCreators, client code, and finally adding a brand-new type to prove the Open/Closed payoff.

नीचे दिया इम्प्लीमेंटेशन इसी अध्ययन दस्तावेज़ के लिए लिखा गया ओरिजिनल कोड है, जो उन्हीं छह चरणों को फ़ॉलो करता है: Product इंटरफ़ेस, ConcreteProducts, एक ऐब्सट्रैक्ट Creator, ConcreteCreators, क्लाइंट कोड, और अंत में Open/Closed फ़ायदा साबित करने के लिए एक बिल्कुल नई टाइप जोड़ना।

Step 1 — The Product Interface

```
<?php

interface Notification
{
    public function send(string $message): string;
}
```

Step 2 — Concrete Products

```
<?php

class EmailNotification implements Notification
{
    public function send(string $message): string
    {
        return "Emailed via SMTP: {$message}";
    }
}

class SmsNotification implements Notification
{
    public function send(string $message): string
    {
        return "Texted via telephony API: {$message}";
    }
}
```

Step 3 — The Abstract Creator

```
<?php

abstract class NotificationCreator
{
    // The factory method – every ConcreteCreator overrides this.
    // हर ConcreteCreator इसे ओवरराइड करता है।
    abstract public function createNotification(): Notification;

    // Shared workflow: doesn't know WHAT it's sending, only HOW to send it.
    // साझा वर्कफ्लो: यह नहीं जानता कि क्या भेज रहा है, सिर्फ यह जानता है कि कैसे भेजना है।
    public function send(string $message): string
    {
        $notification = $this->createNotification();
        return $notification->send($message);
    }
}
```

Step 4 — Concrete Creators

```
<?php

class EmailNotificationCreator extends NotificationCreator
{
    public function createNotification(): Notification
    {
        return new EmailNotification();
    }
}

class SmsNotificationCreator extends NotificationCreator
{
    public function createNotification(): Notification
    {
        return new SmsNotification();
    }
}
```

Step 5 — Client Code

```
<?php

$creators = [
    new EmailNotificationCreator(),
    new SmsNotificationCreator(),
];

foreach ($creators as $creator) {
    echo $creator->send("Welcome!") . "\n";
}
```

Expected output / अपेक्षित आउटपुट:

```
Emailed via SMTP: Welcome!  
Texted via telephony API: Welcome!
```

Step 6 — Adding a New Type (WhatsApp), With Zero Existing Files Touched

```
<?php  
  
class WhatsAppNotification implements Notification  
{  
    public function send(string $message): string  
    {  
        return "WhatsApp message sent: {$message}";  
    }  
}  
  
class WhatsAppNotificationCreator extends NotificationCreator  
{  
    public function createNotification(): Notification  
    {  
        return new WhatsAppNotification();  
    }  
}  
  
// Nothing above this point – Notification, NotificationCreator,  
// EmailNotification, EmailNotificationCreator, etc. – was modified.  
// ऊपर मौजूद कुछ भी – Notification, NotificationCreator,  
// EmailNotification, EmailNotificationCreator, आदि – बदला नहीं गया।  
echo (new WhatsAppNotificationCreator()->send("Your order is confirmed")) . "\n";
```

Expected output / अपेक्षित आउटपुट:

```
WhatsApp message sent: Your order is confirmed
```

6. Practical Example: Document Export System

Here's a second, different-domain scenario making the same point. A reporting service needs to export data in multiple formats — PDF, HTML, CSV — each with its own rendering logic, headers, and file structure, with more formats (Markdown, Excel) plausibly arriving later.

यहाँ एक दूसरा, अलग डोमेन का परिदृश्य (scenario) है जो वही बात साबित करता है। एक रिपोर्टिंग सर्विस को डेटा को कई फ़ॉर्मेट्स में एक्सपोर्ट करना है — PDF, HTML, CSV — हर एक की अपनी रेंडरिंग लॉजिक, हेडर्स, और फ़ाइल संरचना के साथ, और भविष्य में और फ़ॉर्मेट्स (Markdown, Excel) आने की संभावना है।

```

<?php

interface ExportableDocument
{
    public function render(array $rows): string;
}

class PdfDocument implements ExportableDocument
{
    public function render(array $rows): string
    {
        return "[PDF] " . count($rows) . " row(s) rendered with PDF headers/footers.";
    }
}

class HtmlDocument implements ExportableDocument
{
    public function render(array $rows): string
    {
        return "[HTML] <table>" . count($rows) . " row(s)</table>";
    }
}

class CsvDocument implements ExportableDocument
{
    public function render(array $rows): string
    {
        return "[CSV] " . implode(",", array_keys($rows[0] ?? [])) . " + " .
count($rows) . " row(s)";
    }
}

abstract class DocumentExporter
{
    abstract public function createDocument(): ExportableDocument;

    // Shared sequence: header, rows, footer – defined once, here.
    // साझा क्रम: header, rows, footer – यहाँ एक ही बार डिफ़ाइन किया गया।
    public function export(array $rows): string
    {
        $document = $this->createDocument();
        return "Export starting...\n" . $document->render($rows) . "\nExport
complete.";
    }
}

class PdfExporter extends DocumentExporter
{
    public function createDocument(): ExportableDocument
    {
        return new PdfDocument();
    }
}

class HtmlExporter extends DocumentExporter
{
    public function createDocument(): ExportableDocument
    {

```

```

        return new HtmlDocument();
    }
}

class CsvExporter extends DocumentExporter
{
    public function createDocument(): ExportableDocument
    {
        return new CsvDocument();
    }
}

$rows = [
    ['name' => 'Amit', 'total' => 500],
    ['name' => 'Priya', 'total' => 750],
];

foreach ([new PdfExporter(), new HtmlExporter(), new CsvExporter()] as $exporter) {
    echo $exporter->export($rows) . "\n\n";
}

```

What this achieves: adding a new format (say, Markdown) means writing two new classes — nothing else changes, the Open/Closed payoff again; each document type owns only its own formatting logic (Single Responsibility); the shared `export()` sequence (header, rows, footer) is written exactly once, in the base `DocumentExporter`; and every document type can be tested completely independently of the others.

यह क्या हासिल करता है: एक नया फॉर्मेट (जैसे, Markdown) जोड़ने का मतलब है दो नई क्लासेज़ लिखना — और कुछ नहीं बदलता, वही Open/Closed फ़ायदा फिर से; हर document टाइप सिर्फ़ अपनी फॉर्मेटिंग लॉजिक की मालिक है (Single Responsibility); साझा `export()` क्रम (header, rows, footer) बेस `DocumentExporter` में बिल्कुल एक ही बार लिखा गया है; और हर document टाइप को बाक़ी सभी से पूरी तरह स्वतंत्र होकर टेस्ट किया जा सकता है।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Factory Method, Singleton, और Prototype — तीनों क्रिएशनल पैटर्न हैं।
Simple Factory	सिंपल फैक्टरी	<code>SimpleNotificationFactory</code> एक स्टैटिक मेथड में सारा क्रिएशन लॉजिक रखती है — यह GoF पैटर्न नहीं है।
Product Interface	प्रोडक्ट इंटरफ़ेस	<code>Notification</code> इंटरफ़ेस हर नोटिफिकेशन टाइप के लिए कॉन्ट्रैक्ट है।
Concrete Product	कॉन्क्रीट प्रोडक्ट	<code>EmailNotification</code> , <code>Notification</code> इंटरफ़ेस का एक कॉन्क्रीट इम्प्लीमेंटेशन है।
Creator	क्रिएटर	<code>NotificationCreator</code> factory मेथड डिक्लेयर करने वाली ऐब्स्ट्रैक्ट क्लास है।
Concrete Creator	कॉन्क्रीट क्रिएटर	<code>EmailNotificationCreator</code> , <code>createNotification()</code> को ओवरराइड करके <code>EmailNotification</code> लौटाती है।
Factory Method	फैक्टरी मेथड	<code>createNotification()</code> वह मेथड है जिसे हर सबक्लास अपने तरीके से इम्प्लीमेंट करती है।
Open/Closed Principle	ओपन/क्लोज़्ड प्रिंसिपल	नया <code>WhatsAppNotificationCreator</code> जोड़ना, मौजूदा कोड बदले बिना संभव है — यही OCP है।
Decentralized (creation logic)	विकेंद्रीकृत (क्रिएशन लॉजिक)	हर क्रिएटर अपनी खुद की क्रिएशन ज़िम्मेदारी रखता है, एक जगह केंद्रीकृत होने के बजाय।
Delegate (verb)	सौंपना, ज़िम्मेदारी देना	Factory Method, क्रिएशन का फ़ैसला सबक्लासेज़ को सौंप देता है।
Encapsulation	एनकैप्सुलेशन	क्रिएशन लॉजिक को फैक्टरी क्लास के अंदर एनकैप्सुलेट किया जाता है।
Single Responsibility Principle	सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल	हर <code>ExportableDocument</code> सिर्फ़ अपनी फ़ॉर्मेटिंग की ज़िम्मेदारी रखता है।
Template (workflow)	टेम्पलेट (वर्कफ़्लो)	Creator का <code>send()</code> मेथड एक टेम्पलेट की तरह काम करता है — यह "कैसे" जानता है, "क्या" नहीं।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Rigid (figurative)	अकड़ा हुआ, कठोर	"His daily routine was too rigid to allow any last-minute changes." उसकी दिनचर्या इतनी अकड़ी हुई थी कि आखिरी वक़्त में कोई बदलाव नहीं हो सकता था।
Tangled (up)	उलझा हुआ	"The wires behind the TV were completely tangled." टीवी के पीछे तारें पूरी तरह उलझी हुई थीं।
Bloated	बोझिल, फूला हुआ	"The app became bloated after years of adding features without cleanup." सालों तक बिना सफ़ाई के फ़ीचर्स जोड़ते रहने से ऐप बोझिल हो गया।
Eerily (similar)	अजीब तरह से (मिलता-जुलता)	"The new office looked eerily similar to the old one." नया ऑफ़िस पुराने से अजीब तरह से मिलता-जुलता लग रहा था।
Off (something feels off)	गड़बड़, कुछ ठीक नहीं	"Something felt off about the deal, so she walked away." इस डील में कुछ गड़बड़ लगा, इसलिए वह पीछे हट गई।
Versatility	बहुमुखी प्रतिभा, लचीलापन	"The chef's versatility let him cook cuisines from five different countries." शेफ़ की बहुमुखी प्रतिभा से वह पाँच अलग-अलग देशों का खाना बना सकता था।
Manageable	संभालने लायक	"The workload was heavy but still manageable." काम का बोझ ज़्यादा था, पर फिर भी संभालने लायक था।
Plausibly	संभावित रूप से	"The delay was plausibly caused by the storm." यह देरी संभावित रूप से तूफ़ान की वजह से हुई।
Payoff	फ़ायदा, नतीजा	"Years of practice finally had a payoff at the competition." सालों की मेहनत का फ़ायदा आखिरकार प्रतियोगिता में दिखा।
Hand off (verb)	सौंप देना	"She handed off the project to a new manager before leaving." जाने से पहले उसने प्रोजेक्ट एक नए मैनेजर को सौंप दिया।

This document follows the topic order of [algomaster.io's Factory Method lesson](#) (a paid course) but is written independently — original English/Hindi explanations and original PHP code — rather than reproducing their (subscription-gated) material.

यह दस्तावेज़ [algomaster.io](#) के *Factory Method* पाठ (एक पेड कोर्स) के विषय-क्रम को फ़ॉलो करता है, लेकिन स्वतंत्र रूप से लिखा गया है — ओरिजिनल अंग्रेज़ी/हिंदी व्याख्याएँ और ओरिजिनल *PHP* कोड — बजाय इसके कि उनकी (सब्सक्रिप्शन-गेटेड) सामग्री को दोबारा प्रस्तुत किया जाए।